

Queue ADT, Operations & Array Implementation

Q. Define Queue as an Abstract Data Type (ADT). Explain its standard operations and demonstrate its implementation using an Array with a relevant code snippet.

📌 **Definition to Remember**

A **Queue** is a linear data structure that follows the **FIFO (First In, First Out)** principle — the first element inserted is the first to be removed. Insertions occur at the **Rear** (Tail) and deletions occur at the **Front** (Head).

1. Queue as an ADT

As an Abstract Data Type, a Queue manages a collection of elements and restricts access to two specific ends:

- * **Rear:** Where elements are added (enqueued).
- * **Front:** Where elements are removed (dequeued).

2. Standard Operations

Operation	Description	Condition
enqueue(element)	Adds an element to the Rear	Queue Overflow if full
dequeue()	Removes and returns element from Front	Queue Underflow if empty
peek() / front()	Returns Front element without removing	Underflow if empty
isEmpty()	Returns True if queue has no elements	<code>front == -1</code>
isFull()	Returns True if queue is at max capacity	<code>rear == SIZE - 1</code>

3. Array Implementation in C

Two pointers (`front` and `rear`) are initialized to `-1` .

```

#include <stdio.h>
#define SIZE 5

int queue[SIZE];
int front = -1, rear = -1;

void enqueue(int value) {
    if (rear == SIZE - 1) {
        printf("Queue Overflow!\n");
    } else {
        if (front == -1) front = 0; // Set front on first insertion
        queue[++rear] = value;
    }
}

int dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow!\n");
        return -1;
    } else {
        int val = queue[front++];
        // Reset pointers if queue becomes empty
        if (front > rear) front = rear = -1;
        return val;
    }
}

```

4. Advantages and Limitations

- **Advantage:** Array implementation is simple and fast ($O(1)$ time complexity for enqueue/dequeue).
- **Applications:** OS Scheduling (FCFS), IO Buffering (keyboard/printers).
- **Disadvantage (Memory Wastage):** In a simple linear queue, if **rear** reaches the end, we cannot insert new elements — even if spaces at the front were freed by **dequeue()**. (This is solved by Circular Queues).

★ Must-Write Points (for 10 marks)

1. Queue follows the **FIFO (First In, First Out)** principle.
2. Elements are inserted at the **Rear** and removed from the **Front**.
3. Core operations: **enqueue (insert)** and **dequeue (remove)**.
4. **Queue Overflow** occurs if enqueue is called when full (**rear == SIZE - 1**).
5. **Queue Underflow** occurs if dequeue is called when empty (**front == -1**).
6. In array implementation, two pointers are maintained: **front** and **rear**, both initialized to -1.
7. Linear array queues waste memory — empty front spaces cannot be reused once rear hits the end.

⚡ Quick Recall

Queue → FIFO → Insert at Rear (enqueue) → Remove from Front (dequeue) → Overflow (rear==SIZE-1) → Underflow (front==-1) → Linear array wastes freed memory



Complete Executable C Code: Linear Queue Array Implementation

```
#include <stdio.h>

#define MAX 5

struct Queue {
    int items[MAX];
    int front;
    int rear;
};

void initQueue(struct Queue *q) {
    q->front = -1;
    q->rear = -1;
}

int isFull(struct Queue *q) {
    return q->rear == MAX - 1;
}

int isEmpty(struct Queue *q) {
    return q->front == -1 || q->front > q->rear;
}

void enqueue(struct Queue *q, int val) {
    if (isFull(q)) {
        printf("Queue Overflow! Cannot enqueue %d\n", val);
        return;
    }
    if (q->front == -1) q->front = 0;
    q->items[++(q->rear)] = val;
    printf("Enqueued: %d\n", val);
}

int dequeue(struct Queue *q) {
    if (isEmpty(q)) {
        printf("Queue Underflow! Cannot dequeue\n");
        return -1;
    }
    int val = q->items[q->front++];
    if (q->front > q->rear) { // Reset queue when empty
        q->front = -1;
        q->rear = -1;
    }
    return val;
}

void display(struct Queue *q) {
    if (isEmpty(q)) {
        printf("Queue is Empty\n");
        return;
    }
    printf("Queue (Front to Rear): ");
    for (int i = q->front; i <= q->rear; i++) {
        printf("%d ", q->items[i]);
    }
}
```

```
    }  
    printf("\n");  
}  
  
int main() {  
    struct Queue q;  
    initQueue(&q);  
  
    enqueue(&q, 10);  
    enqueue(&q, 20);  
    enqueue(&q, 30);  
    display(&q);  
  
    printf("Dequeued: %d\n", dequeue(&q));  
    display(&q);  
  
    return 0;  
}
```